WhatsApp Volunteer Bot

https://gemini.google.com/app/7f05cfba07c8018f

User prompt: I have a pressingidea that might improve Volunteer management at the space. Foor a long time our coordinator has been struggling to find an effiecient way of handling tasks. Other volunteers have created platfroms/apps/websites for that but they never seem to catch on. Recently a Notion page was to be set up, which in itself is an amazing idea. But I asked myself, why do they never catch on? And the answer was always in me all along. I don't like downloading,installing new apps or accessing certain websites to get work done. It feels fragmented and you do it one week, and then you eventually stop. Why not bring the solution closer: WhatsApp. I have been thinking of developing a whatsapp bot for American Corner Mtaani. One that will be in our groups that will always be used to assign tasks and track them. The idea is not fully defined but I needed to get it out first and built it out in phases. With my first phase being able to assign tasks. My coordinator can just use a special tag and just talk as normal as he does assigning tasks. This special tag triggers the bot. In the same message where he's assign the task he tags the individual. The bot takes the task, name, and time. It is stored, and then the bot sends a WhatsApp message to the volunteer about the task and if it has a deadline. The bot can also be used to track progress by asking about progress close to the deadline. This is the first phase. Later it can be used to inquire general information like SOP's, certain important links that volunteers need to have but only that. This will not be a general purpose chat bot for now, so that we focus on the problem, volunteers can also use it to log time and activities at the space. "What time the entered and what time they left." I first want to know how to go about it and the resources required. I plan on building it out with antigravity, but I don't just want to vibe code, I want to be taken through every step to understand and know how the building process happens and why it works. I also want

to figure out where the information collected by the chatbot will be collected. It is still nice to have a document/page for reference, all documents should remain similar.

Response:

User prompt: I have a pressingidea that might improve Volunteer management at the space. Foor a long time our coordinator has been struggling to find an effiecient way of handling tasks. Other volunteers have created platfroms/apps/websites for that but they never seem to catch on. Recently a Notion page was to be set up, which in itself is an amazing idea. But I asked myself, why do they never catch on? And the answer was always in me all along. I don't like downloading,installing new apps or accessing certain websites to get work done. It feels fragmented and you do it one week, and then you eventually stop. Why not bring the solution closer: WhatsApp. I have been thinking of developing a whatsapp bot for American Corner Mtaani. One that will be in our groups that will always be used to assign tasks and track them. The idea is not fully defined but I needed to get it out first and built it out in phases. With my first phase being able to assign tasks. My coordinator can just use a special tag and just talk as normal as he does assigning tasks. This special tag triggers the bot. In the same message where he's assign the task he tags the individual. The bot takes the task, name, and time. It is stored, and then the bot sends a WhatsApp message to the volunteer about the task and if it has a deadline. The bot can also be used to track progress by asking about progress close to the deadline. This is the first phase. Later it can be used to inquire general information like SOP's, certain important links that volunteers need to have but only that. This will not be a general purpose chat bot for now, so that we focus on the problem, volunteers can also use it to log time and activities at the space. "What time the entered and what time they left." I first want to

know how to go about it and the resources required. I plan on building it out with antigravity, but I don't just want to vibe code, I want to be taken through every step to understand and know how the building process happens and why it works. I also want to figure out where the information collected by the chatbot will be collected. It is still nice to have a document/page for reference, all documents should remain similar.

Response: Meeting volunteers where they already communicate is the most effective way to eliminate adoption friction. Building a WhatsApp bot for American Corner Mtaani ensures the tool integrates directly into existing workflows without requiring new accounts or downloads.

To execute this in Python while maintaining a clear, readable reference point for the coordinator, here is the architecture and the resources required.

Core Resources & Tech Stack

- **WhatsApp Integration:** Meta WhatsApp Cloud API. It allows you to receive webhooks from group messages and send outbound direct messages.
- **Backend Logic:** Python with FastAPI or Flask. This will serve as the webhook listener and the central brain of the operation.
- **Data Extraction:** Google Gemini API. Since the coordinator will assign tasks in natural language, Gemini can extract the specific task, assignee name, and deadline into structured JSON data.
- **Database / Storage:** Google Sheets. This acts as the perfect familiar dashboard for the coordinator. It provides a static reference document that dynamically updates as the bot processes tasks.

- **Hosting:** Ngrok for local testing, transitioning to a lightweight VPS or cloud function for deployment.

The Step-by-Step Building Process

1. **The Webhook Setup:** You build a local Python server that acts as a listener. When a message containing your specific trigger (e.g., "@MtaaniBot") is sent in the WhatsApp group, Meta's API pushes that message payload to your server.

2. **Message Parsing:** Your script extracts the text payload and passes it to the Gemini API. You will design a strict system prompt instructing the model to parse the conversational text and return only the `assignee`, `task_description`, and `deadline`.

3. **Database Logging:** Using the Google Sheets API, your Python script takes the parsed data and appends it as a new row in a designated tracker sheet, capturing the timestamp, task, assignee, and status.

4. **Outbound Notification:** Once the database logs the entry, your script makes a POST request back to the WhatsApp API to send a direct, formatted message to the assignee's specific phone number with their task details.

5. **Automated Follow-ups:** A background scheduler (like APScheduler or a cron job) runs continuously, scanning the Google Sheet for approaching deadlines and triggering reminder messages to the respective volunteers.

Which piece of this architecture do you want to tackle first—setting up the WhatsApp API webhook to receive messages, or designing the data extraction script?

User prompt: I want the path that costs me near zero or zero dollars

Response: To build this for zero dollars while supporting standard WhatsApp groups, you must bypass the official Meta Cloud API, which restricts group bots and charges for volume. Here is the completely free Python architecture.

The Zero-Dollar Stack

- **WhatsApp Gateway: Green API.** This acts as a wrapper for WhatsApp Web. You scan a QR code with a phone to create an API endpoint. The free Developer Tier limits you to interacting with 3 specific chats—perfect for the Mtaani group, your personal testing number, and the coordinator.
- **Data Extraction: Google Gemini API.** The free tier allows 15 requests per minute, easily handling the natural language parsing to extract the assignee, task, and deadline.
- **Database: Google Sheets API.** 100% free, acts as a visual dashboard, and connects natively to Python.
- **Hosting: Render + UptimeRobot.** You can host your Python (FastAPI or Flask) webhook listener on Render's free tier. Render spins down inactive apps after 15 minutes, so you use UptimeRobot (also free) to ping it every 14 minutes to keep the bot awake.

The Execution Flow

1. The coordinator messages the group: *"@MtaaniBot, assign @VolunteerName to organize the VR headsets by Friday."*

2. Green API catches the message and sends a webhook to your Render server.

3. Your script passes the text to Gemini, which returns structured JSON data.

4. Python appends the row to Google Sheets, then commands Green API to message the volunteer.

Sources

1. WhatsApp API — Plans and Pricing
